

Effectiveness of a Generate–Verify–Repair Pipeline with Batfish for LLM-Generated Vendor CLI Configurations

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We report a preregistered paired experiment (cnsms2026-001-gvr-batfish-prereg-v1) that evaluated whether a generate–verify–repair (GVR) pipeline—shared initial LLM candidate (gpt-5-mini) + a deterministic Batfish-style validator + at most one automated LLM repair—reduces deterministic-verifier detected semantic/safety failures on intent→vendor-CLI tasks. Planned N=300 pairs; 266 complete pairs were analysed after the preregistered ‘complete_pair_primary’ missingness rule. Baseline pass-rate = 260/266 (97.744%); guarded (final GVR) pass-rate = 265/266 (99.624%). Paired difference = 0.01879699248; exact McNemar two-sided $p = 0.0625$. The exact McNemar two-sided test was the preregistered primary hypothesis test and decision rule (preregistration manifest SHA-256:). A paired bootstrap (B=10,000, seed=1729) percentile 95% CI = [0.0037593984962406013, 0.03759398496240601] is reported as a supplementary/descriptive uncertainty summary and not used to overturn the preregistered test decision. Repair was invoked for 6 tasks; median repair iterations per task = 0. Per-episode latency was not recorded and thus the preregistered latency estimand could not be evaluated. Under shared_initial_candidate semantics the observed absolute improvement is small and did not meet the preregistered $\alpha=0.05$ decision rule. We archive exact paired counts, provider/stage accounting, representative artifact-grounded repair examples, and reconciliation artifacts to support audit and reanalysis.

I. INTRODUCTION

Generative large language models (LLMs) are actively explored for NetOps tasks such as intent→CLI translation, zero-touch configuration synthesis, and automated maintenance workflows [1], and surveys emphasize both opportunity and operational risk when LLMs are applied to network configuration [2]. Stochastic LLM outputs can produce syntactic or semantic violations—missing required commands, unsafe command orderings, or constraint breaches—that create real operational hazards if applied directly. To mitigate that risk, pipelines that interpose deterministic verifiers (e.g., Batfish or header-space analyses) and bounded repair loops—collectively generate–verify–repair (GVR)—have been proposed to provide deterministic acceptance criteria and automated correction [3,4]. Prior work describes components and prototypes, but preregistered, paired evaluations that archive verifier traces and quantify verifier-triggered repair benefit on reproducible NetOps task banks remain limited.

This study tested the preregistered question (cnsms2026-001-gvr-batfish-prereg-v1): Does a GVR pipeline that uses a single sampled initial LLM candidate per task (shared_initial_candidate semantics), a deterministic

Batfish-style validator, and at most one automated LLM repair call reduce deterministic-verifier detected semantic/safety failures relative to the same initial candidate without repair? Our principal contribution is a preregistered, artifact-archived paired evaluation executed under the CNSM Agentic AI Researcher constraints that reports exact paired counts, inferential outcomes using the preregistered decision rule, provider/stage accounting, representative repaired examples, and an evidence-grounded interpretation.

II. RELATED WORK

LLM applications in NetOps span intent-to-configuration translation, automated configuration generation, and AIOps toolchains; several recent benchmark and survey efforts document these applications and their limitations [1,2]. Benchmarks such as NetConfEval assess whether LLMs can synthesize vendor CLI fragments from intent descriptions and illustrate that vendor-specific syntax, sequencing requirements, and implicit ordering constraints remain failure points for unconstrained generation [1]. Surveys of LLMs for network operations synthesize common failure modes—hallucination, constraint violations, and brittle prompt dependence—and recommend grounding, verifier integration, and tooling to reduce operational risk [2].

Generate–verify–repair (GVR) architectures have been proposed in adjacent domains (e.g., code generation and regulated document production) to combine a stochastic generator with a deterministic oracle and bounded repair steps; these works provide architectures and convergence desiderata for achieving reproducible, verifiable outputs from stochastic agents [3]. In NetOps, deterministic network verifiers (Batfish and related analyses) provide concrete, automatable checks for reachability, ordering, and policy constraints; Batfish’s engineering history highlights tradeoffs in rule expressiveness and scalability that directly affect its suitability as a validator in automated loops [4]. Empirical studies of LLMs synthesizing router/device configurations emphasize that correct sequencing, vendor dialect handling, and explicit constraint encoding are necessary for high first-pass correctness and motivate verifier-driven repair when those elements are missing [5].

Our work differs from prior work by executing a preregistered paired test using shared_initial_candidate semantics, by archiving raw model and validator traces for auditability, and by reporting exact paired contingency counts together with

the preregistered McNemar test and a supplementary paired bootstrap CI to aid interpretation.

III. METHODOLOGY

Preregistration and estimand: The experiment followed preregistration `cnsms2026-001-gvr-batfish-prereg-v1` (manifest SHA-256:). The primary estimand is the `paired_success_rate_difference_guarded_minus_baseline` under `shared_initial_candidate` semantics. Under these semantics a single sampled initial model candidate per task is used to produce both outcomes: (1) baseline — score the single initial candidate with the deterministic validator; (2) guarded — if that same candidate fails the validator, the pipeline may invoke at most one automated LLM repair call and then rescore with the same deterministic validator. This design isolates the verifier-triggered repair effect for the observed initial draw and matches the preregistered `execution_contract` (`generation_semantics = "shared_initial_candidate"`).

Task bank, sampling, and inclusion rules: The task bank was deterministically generated to produce $N=300$ intent→CLI pairs, stratified into two vendor-dialect clusters (150 Cisco-like; 150 Juniper-like) via a seeded synthetic generator supplemented with public NetConfEval-style items as specified in the preregistration. Inclusion/exclusion rules followed the preregistered contract: exact public duplicates detected by contamination checks are excluded from the primary confirmatory analysis (none were excluded for exact duplication in this run); provider/API transient failures triggered up to two automatic retries; pairs where both arms failed due to infrastructure/model API errors were excluded from the primary paired analysis per the `'complete_pair_primary'` missingness rule. The deterministic reconciliation artifact records these exclusions (see `deterministic_reconciliation.json` SHA-256:).

Model orchestration and runtime configuration: The declared model used for generation and any repair calls was `gpt-5-mini` (the archived model configuration `declared_model_version = "gpt-5-mini"`). Archived configuration fields include `max_output_tokens = 1024`, `maximum_attempts_per_call = 1`, `provider = openai_responses`, and `temperature = null` (unset). Provider accounting recorded `hosted_model_call_event_count = 306`, `completed_hosted_model_call_event_count = 272`, and `failed_hosted_model_call_event_count = 34`; stage accounting recorded `shared_initial_generation = 300` and `repair = 6`. Because temperature was unset and no centralized deterministic sampling seed for provider calls is archived, nondeterministic sampling of provider responses cannot be excluded for the recorded calls; this runtime nondeterminism is declared and discussed in Limitations and the Disclosure Statement.

Provider/stage accounting semantics (explicit clarification): under `shared_initial_candidate` semantics the pipeline issues exactly one shared initial model generation per task (recorded as `shared_initial_generation = 300`). That single sampled candidate is material to both arms: the baseline decision is the validator score of the shared candidate; the guarded outcome

is the final validator score after rescoring that same candidate unless the validator failed and the bounded repair stage was invoked. The recorded repair count (`repair = 6`) enumerates additional model calls invoked only when the shared initial candidate failed. Provider `'condition_counts'` and `'stage_counts'` in the execution manifest therefore reflect model-call stages (shared initial calls and repair calls) and not independent full-generation arms per condition; provider `call_audit` totals for this run were `hosted_model_call_event_count = 306`, `completed = 272`, `failed = 34`, `stage_counts: {shared_initial_generation: 300, repair: 6}`, and `condition_counts: {shared: 300, guarded: 6}` (these counts are archived in `deterministic_reconciliation.json`).

Deterministic validator and scoring: The binary oracle `deterministic_netops_validator_v5` executed an archived fixed rule battery per episode: (1) CLI syntactic parsing and normalization; (2) required-sequence checks against the task's `required_sequence`; (3) constrained-field touch checks versus `allowed_touched_fields`; and (4) reachability/constraint validation over a deterministic synthetic topology (a Batfish-style evaluation). Validator traces archive `parsed_command_count`, `required_command_count`, `observed_touched_field_count`, violation codes, `normalized_configuration`, and a boolean valid flag. The primary binary endpoint used the preregistered scoring rule `'full_intent_constraint_validation'` recorded in scoring artifacts.

Repair loop mechanics and bounding: Per preregistration the automated repair stage was bounded to at most one LLM repair call per episode when the initial candidate failed the validator. Repair prompt construction (archived) included the task constraints, validator failure codes, and explicit instruction to produce a corrected configuration satisfying the same task constraints. Six repair calls were executed in this run; repair prompts and repaired outputs are archived with subsequent validator traces for audit.

Analysis plan and inferential procedures (preregistered primary decision rule): The preregistered primary hypothesis test is the exact McNemar two-sided test on paired binary outcomes at $\alpha=0.05$ (estimator id `paired_success_rate_difference_guarded_minus_baseline`). Paired bootstrap percentile CIs ($B=10,000$, $\text{seed}=1729$) were specified for descriptive interval estimation of the paired difference and are reported as supplementary/descriptive summaries; they are not the preregistered primary decision procedure and should not be used to overturn the preregistered McNemar decision. Missingness handling followed the preregistered `'complete_pair_primary'` rule: pairs with both-arm provider/infrastructure failures were excluded from the confirmatory McNemar test (these exclusions are recorded in `deterministic_reconciliation.json`; SHA-256:). Secondary analyses (median repair iterations, median latency) were preregistered; per-episode latency was not archived and therefore the latency estimand could not be evaluated.

IV. RESULTS

Execution accounting and sample flow: The preregistered plan targeted 300 independent intent→CLI tasks (600 episodes across two conditions). The run recorded `planned_episode_count` = 600 and `completed_episode_count` = 532; provider/infrastructure transient failures produced `failed_episode_count` = 68. Applying the preregistered 'complete_pair_primary' exclusion rules (exclude both-arm provider/infrastructure failures) yielded complete paired units analysed = 266. Provider accounting indicates `hosted_model_call_event_count` = 306 total model-call events, with `completed_hosted_model_call_event_count` = 272 and `failed_hosted_model_call_event_count` = 34; stage accounting records `shared_initial_generation` = 300 and repair = 6.

Primary paired outcomes and contingency counts: For the 266 complete pairs the validator outcomes produced contingency counts: n_{11} = 260 (both baseline and guarded pass), n_{10} = 5 (guarded pass only), n_{01} = 0 (baseline pass only), and n_{00} = 1 (both fail). Observed per-condition proportions: baseline = $260/266 = 0.9774436090$ and guarded = $265/266 = 0.9962406015$; paired (guarded - baseline) difference = 0.01879699248. The preregistered exact McNemar two-sided test on the discordant counts (n_{10} = 5, n_{01} = 0) produced p = 0.0625 and therefore did not reject the null at α = 0.05.

Bootstrap CI and interpretive note: The paired bootstrap percentile CI (B = 10,000 resamples; seed = 1729) for the paired difference is [0.0037593985, 0.0375939850]. This interval is presented as a supplementary descriptive measure of effect magnitude and resampling variability; it is not used to overturn the preregistered McNemar decision. The discrete exact McNemar p is conservative with few discordants; reporting both statistics provides complementary perspectives on uncertainty for small-discordant outcomes.

Repair invocation and yield: Repair was invoked in 6 episodes. The discordant pattern (n_{10} = 5) indicates that at most five successful repairs contributed to additional guarded successes among analysed pairs; at most one repair among the analysed pairs did not convert to a guarded pass. Median repair iterations per analysed pair = 0 (repairs were infrequent). A descriptive number-needed-to-treat (NNT) interpretation is $1 / 0.01879699248 \approx 53.2$ —roughly 53 guarded tasks were associated with one additional verifier-approved configuration under this run's conditions and `shared_initial_candidate` semantics—reported as a contextual operational scalar, not an operational claim.

Representative repair case diagnostics: Representative artifacts illustrate typical failure and repair behavior. Task-000008 (difficulty: "extreme") initially produced a truncated command that failed `parsed_command_count` vs `required_command_count` checks; the automated repair call produced a corrected candidate that completed the missing line and satisfied the validator. Per-episode validator traces archive `parsed_command_count`, `required_command_count`, violation codes, and `normalized_configuration` to support inspection of

repaired episodes.

Sensitivity to missingness: Thirty-four both-arm episodes were excluded from the primary confirmatory analysis per the preregistered 'complete_pair_primary' rule; this exclusion and the reconciliation of episode accounting are recorded in `deterministic_reconciliation.json` (SHA-256:). A preregistered conservative sensitivity analysis that treats those 34 excluded both-arm episodes as failures yields baseline = $260/300 = 0.866667$ and guarded = $265/300 = 0.883333$; the discordant counts among the complete pairs remain n_{10} = 5, n_{01} = 0 and the exact McNemar p for the analysed pairs remains 0.0625.

Reproducibility metadata: All numerical summaries derive from archived artifacts: the archived paired contingency table ($n_{11}, n_{10}, n_{01}, n_{00}$), the archived condition summary, the archived analysis artifact, and provider accounting in `deterministic_reconciliation.json`. The paired bootstrap used B = 10,000 resamples with seed = 1729 (archived in `analysis_log.jsonl`). Raw responses, provider traces, validator traces, and scoring JSONs are archived and hashed to permit exact reanalysis.

V. DISCUSSION

Statistical interpretation and small-discordant behavior: The guarded GVR pipeline produced a small absolute improvement (~1.88 percentage points). The preregistered exact McNemar test did not reject at α =0.05 (p =0.0625). The paired bootstrap CI that excludes zero highlights finite-sample discreteness: with few discordants (n_{10} =5, n_{01} =0) McNemar's discrete two-sided p -value can be conservative while bootstrap percentile CIs reflect empirical resampling variability. We present both inferential perspectives and state explicitly that the exact McNemar test is the preregistered primary decision rule (preregistration SHA-256:) and that the bootstrap CI is supplementary.

Operational implications and boundary conditions grounded in artifacts: Baseline first-pass accuracy was high (~97.7%), leaving few failures for the repair stage to correct—hence rare repair invocations (6/300). Stage accounting and provider traces show repair-stage overhead was small in the observed run, but per-episode latency was not archived and thus end-to-end timing against the preregistered operational bound could not be measured. The low repair rate suggests bounded automated repair adds limited marginal benefit on high-first-pass corpora; GVR may yield larger absolute improvements on corpora with lower first-pass accuracy or different sampling regimes.

Threats to validity (artifact-grounded): Internal validity: `shared_initial_candidate` semantics measures the effect of validator-triggered repair for a fixed initial sample and does not assess independent multiple draws or constrained first-pass prompting because independent first-pass arms were not executed. Construct validity: `deterministic_netops_validator_v5` enforces a finite, archived rule battery and therefore semantic violations outside that battery are undetected and unmeasured. External validity: tasks derive from public NetConfEval-style examples and a seeded synthetic generator limited to two vendor dialects; results may not generalize to other vendors,

larger topologies, or production heterogeneous workloads. Conclusion validity: the loss of 34 both-arm provider failures reduced effective N and few discordant pairs limit power for small effect detection.

Design lessons and recommendations grounded in execution artifacts: (1) When baseline pass rates are high, target corpora with lower first-pass accuracy or include independent multiple-draw arms to increase discordant counts and power. (2) Archive per-episode timing (generation→validation→repair→final validation) in future runs to assess latency estimands (this run lacked per-episode timing). (3) Publish and expand validator rule coverage and include tests that exercise potential verifier false negatives; per-episode validator traces permit such audits. (4) Consider separate preregistered arms comparing `shared_initial_candidate` semantics to independent-generation semantics if the research question requires measuring other interventions.

VI. LIMITATIONS

- `Shared_initial_candidate` semantics: both arms used the same single sampled initial candidate per preregistration; the estimand measures validator-triggered repair benefit for that fixed initial sample and does not evaluate independent first-pass generations or constrained first-pass prompting strategies.
- Missingness and sample reduction: planned N=300 pairs; after infrastructure/provider exclusions 266 pairs were complete and 34 both-arm episodes were excluded per preregistered rules, reducing effective sample size and precision (see `deterministic_reconciliation.json` SHA-256:).
- Small discordant counts: primary inference used exact McNemar with few discordants ($n_{10}=5$, $n_{01}=0$); conclusions are sensitive to inferential choice and limited power.
- Validator coverage: `deterministic_netops_validator_v5` enforces a finite set of syntactic/semantic/reachability rules; semantic violations outside its coverage are possible and unmeasured.
- Runtime nondeterminism and sampling: the archived model configuration records `temperature = null` and no deterministic seed; nondeterministic sampling cannot be excluded for recorded model calls.
- H2 latency not evaluated: per-episode end-to-end latency was not present in archived per-task artifacts and therefore the preregistered latency bound (median ≤ 60 s) could not be assessed.
- External validity: task corpus derives from public NetConfEval-style examples and a seeded synthetic generator limited to two vendor-dialect clusters; results do not imply universal benefit across other vendors, larger topologies, or production deployments.

VII. CONCLUSION

We executed a preregistered paired experiment that evaluated a GVR pipeline (`gpt-5-mini` +

`deterministic_netops_validator_v5` + ≤ 1 automated repair) on intent→CLI tasks under `shared_initial_candidate` semantics. Across 266 analysed pairs the guarded pipeline improved validator pass rate by 0.01879699248 but did not meet the preregistered exact-McNemar $\alpha=0.05$ decision rule ($p=0.0625$); a paired bootstrap CI ($B=10,000$, $\text{seed}=1729$) is reported as a supplementary descriptive interval and narrowly excludes zero. Repair calls were rare (6/300) and median repair iterations per analysed pair = 0. Per-episode latency was not archived and thus the preregistered latency estimand could not be evaluated. All execution and analysis artifacts (raw responses, validator traces, provider call accounting, and reconciliation manifests) are archived to support reanalysis and follow-on work.

DISCLOSURE STATEMENT

Disclosure (concise): Declared model: `gpt-5-mini` (the archived model configuration archived; `declared_model_version = "gpt-5-mini"`). Orchestration adapter: `hosted_netops_gvr_v1`. Deterministic validator: `deterministic_netops_validator_v5`. Preregistration: `cnsms2026-001-gvr-batfish-prereg-v1` (manifest SHA-256:). Initial master prompt immutable reference: `provenance/master_prompt.txt` (SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582b39b15ba). Human role and autonomy boundary: before the autonomy lock a human Research Director selected the topic family and approved pre-lock infrastructure development; after the autonomy lock the registered pipeline autonomously performed literature selection, preregistration, experimental execution, analysis, manuscript drafting, AI peer review simulation, and revision. Provider accounting (archived) recorded `hosted_model_call_event_count = 306`, `completed_hosted_model_call_event_count = 272`, `failed_hosted_model_call_event_count = 34`; stage accounting recorded `shared_initial_generation = 300` and `repair = 6`. Nondeterminism disclosure: the archived model configuration records `temperature = null` (unset) and no deterministic sampling seed; therefore nondeterministic sampling of model outputs cannot be excluded for recorded provider calls. Execution and analysis artifacts, logs, and hash manifests are archived in the evidence bundle referenced in the preregistration.

REFERENCES

- [1] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, Marco Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?", 2024, 10.1145/3656296
- [2] S.Y. Long, Jingjing Tan, Bomin Mao, Fengxiao Tang, Yangfan Li, Ming Zhao, Nei Kato, "A Survey on Intelligent Network Operations and Performance Optimization Based on Large Language Models", 2025, 10.1109/comst.2025.3526606
- [3] Rafael Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments", 2026, 10.2139/ssrn.6754899
- [4] Matt Brown, Ari Fogel, Daniel Halperin, Victor Heorhiadi, Ratul Mahajan, Todd Millstein, "Lessons from the evolution of the Batfish configuration analysis tool", 2023, 10.1145/3603269.3604866

- [5] Rajdeep Mondal, Alan Tang, Ryan Beckett, Todd Millstein, George Varghese, "What do LLMs need to Synthesize Correct Router Configurations?", 2023, 10.1145/3626111.3628194